Generative AI Curriculum

Module 3: Autoencoders and GANs

Ultra-Detailed Guide to Advanced Generative Models

PART 1: AUTOENCODERS

What is an Autoencoder?

Autoencoder = Neural network trained to compress and decompress data

Architecture:

```
Input (High-Dimensional)
  ↓ Encoder (Compression)
  ↓ Latent Representation (Low-Dimensional)
  ↓ Decoder (Reconstruction)
  ↓ Output (Attempts to match input)
```

Loss Function:

$$L = ||x - \hat{x}||^2$$

Where:

- x = original input
- $\hat{x}$ = reconstructed output
- Goal: Minimize reconstruction error

Why Autoencoders?

Key Insight: If network successfully compresses and reconstructs, the compressed representation must capture essential features.

Applications:

- Dimensionality reduction (like PCA but non-linear)
- Feature extraction
- Data compression
- Anomaly detection
- Denoising

Simple Autoencoder Architecture

Example: Image Compression

```
Input Image: 28×28×1 = 784 pixels
  ↓
Encoder:
  Dense(512) → ReLU
  Dense(256) → ReLU
  Dense(32) → ReLU  (Latent: 32D)
  ↓
Latent Space: 32 dimensions (24.5× compression!)
  ↓
Decoder:
  Dense(256) → ReLU
  Dense(512) → ReLU
  Dense(784) → Sigmoid
  ↓
Output Image: 28×28×1 = 784 pixels
```

Training Process

Step 1: Forward pass through autoencoder

```
Input → Encoder → Latent (z) → Decoder → Output
```

Step 2: Compute reconstruction loss

```
Loss = MSE(input, output)
```

Step 3: Backpropagation

```
Update encoder weights
Update decoder weights

Goal: Minimize reconstruction error
```

Implementation

```
import tensorflow as tf
from tensorflow import keras

# Build autoencoder
encoder = keras.Sequential([
    keras.layers.Dense(512, activation='relu', input_shape=(784,)),
    keras.layers.Dense(256, activation='relu'),
    keras.layers.Dense(32, activation='relu')  # Latent layer
])
```

```
decoder = keras.Sequential([
    keras.layers.Dense(256, activation='relu', input_shape=(32,)),
    keras.layers.Dense(512, activation='relu'),
    keras.layers.Dense(784, activation='sigmoid')
])

# Combine
autoencoder = keras.Sequential([encoder, decoder])
autoencoder.compile(optimizer='adam', loss='mse')

# Train
autoencoder.fit(x_train, x_train, epochs=50, batch_size=32)

# Evaluate reconstruction
reconstructed = autoencoder.predict(x_test)
mse = np.mean((x_test - reconstructed) ** 2)
```

Applications of Standard Autoencoders

1. Anomaly Detection

```
Normal samples: Small reconstruction error
Anomalies: Large reconstruction error

Threshold: If error &gt; threshold → Anomaly detected

Detection Rate: 95% with proper tuning
```

2. Denoising

```
Input: Noisy image
Trained On: Clean images
Latent Space: Filters out noise
Output: Denoised image

Use: Image restoration, signal processing
```

3. Feature Extraction

```
Pretrain Autoencoder on unlabeled data
Extract encoder
Use latent features for downstream tasks

Result: Better features than random initialization
```

Variational Autoencoders (VAE)

Problem with Standard Autoencoders

```
Standard AE Latent Space:
- Learns compressed representation
- No structure or meaning
- Can't generate new data easily
- Random point in latent space → Garbage

Example:
  Random z = [0.5, -0.2, 0.8, ...]
  Decode → Meaningless image (not a real digit)
```

Solution: Variational Structure

Key Idea: Make latent space follow a known distribution (Gaussian)

Architecture:

```
Encoder outputs:
  - Mean (μ) of latent distribution
  - Log-variance (log σ²)

Sampling:
  z = μ + σ * ε  where ε ~ N(0, I)
  (Reparameterization trick)

Decoder:
  Generate from z
```

VAE Loss Function

Total Loss = Reconstruction Loss + KL Divergence

$$L = MSE(x, \hat{x}) + KL(q(z|x)||p(z))$$

Reconstruction Loss: How well we reconstruct

$$L_{recon} = \frac{1}{N}\sum_{i=1}^{N}||x_i - \hat{x}_i||^2$$

KL Divergence: How close latent distribution is to standard Gaussian

$$L_{KL} = -\frac{1}{2}\sum_{j=1}^{J}(1 + \log\sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

Intuition:

- High reconstruction loss: Increase λ_recon

- KL divergence too high: Decrease λ_KL

VAE Implementation

```
from tensorflow.keras.layers import Input, Dense, Lambda
from tensorflow.keras import backend as K
import numpy as np

# Parameters
latent_dim = 32
batch_size = 32

# Sampling layer (reparameterization trick)
def sampling(args):
    z_mean, z_log_var = args
    epsilon = K.random_normal(shape=(K.shape(z_mean)[^0], latent_dim))
    return z_mean + K.exp(0.5 * z_log_var) * epsilon

# Encoder
inputs = Input(shape=(784,))
x = Dense(512, activation='relu')(inputs)
x = Dense(256, activation='relu')(x)
z_mean = Dense(latent_dim)(x)
z_log_var = Dense(latent_dim)(x)
z = Lambda(sampling, output_shape=(latent_dim,))([z_mean, z_log_var])

encoder = keras.Model(inputs, [z_mean, z_log_var, z])

# Decoder
latent_inputs = Input(shape=(latent_dim,))
x = Dense(256, activation='relu')(latent_inputs)
x = Dense(512, activation='relu')(x)
outputs = Dense(784, activation='sigmoid')(x)

decoder = keras.Model(latent_inputs, outputs)

# VAE
z_output = encoder(inputs)[^2]
reconstructed = decoder(z_output)

# Custom loss
def vae_loss(x, x_reconstructed):
    # Reconstruction loss
    recon_loss = K.binary_crossentropy(x, x_reconstructed)
    recon_loss *= 784

    # KL divergence
    kl_loss = 1 + z_log_var - K.square(z_mean) - K.exp(z_log_var)
    kl_loss = K.sum(kl_loss, axis=-1)
    kl_loss *= -0.5

    return K.mean(recon_loss + kl_loss)

vae = keras.Model(inputs, reconstructed)
```

```
vae.compile(optimizer='adam', loss=vae_loss)
vae.fit(x_train, x_train, epochs=50)
```

VAE Properties

1. Continuous Latent Space

```
z1 = encode(digit_2)
z2 = encode(digit_5)
z_blend = 0.5 * z1 + 0.5 * z2

decode(z_blend) = Interpolated between 2 and 5!
Smooth morphing effect
```

2. Generative Capability

```
z ~ N(0, I)  (Sample from standard Gaussian)
decode(z) → Generated digit

Result: Generate infinite new, realistic samples
```

3. Disentangled Representation

```
Special case: β-VAE

Loss = recon_loss + β * KL_divergence
β &gt; 1: Stronger regularization
β &lt; 1: Weaker regularization

Result: Latent dimensions learn interpretable factors
  e.g., dimension 1 = thickness
        dimension 2 = slant
        dimension 3 = brightness
```

PART 2: GENERATIVE ADVERSARIAL NETWORKS (GANs)

What is a GAN?

GAN = Two networks compete: Generator creates, Discriminator judges

Architecture:

```
Generator:
  Random noise → Generates fake images
  Goal: Fool discriminator

Discriminator:
  Real images + Fake images → Classify real/fake
```

```
  Goal: Correctly distinguish

Competition:
  Generator tries harder to fool
  Discriminator tries harder to detect
  → Both improve through adversarial process
```

Game Theory Perspective

Zero-Sum Game:

$$\min_G \max_D E_x[\log D(x)] + E_z[\log(1 - D(G(z)))]$$

Nash Equilibrium: Neither can improve further

- D accuracy = 50% (can't distinguish)
- Generator distribution matches real distribution

GAN Training Dynamics

Step 1: Train Discriminator

```
Discriminator sees:
  - Real images: Label = 1
  - Fake images from generator: Label = 0

Loss: Binary cross-entropy
  L_D = -[log(D(real)) + log(1 - D(fake))]

Update: Improve discriminator weights
```

Step 2: Train Generator

```
Generator wants to fool discriminator:
  L_G = -log(D(G(z)))

Alternative (more stable):
  L_G = -log(D(G(z))) (same as D loss)

Update: Improve generator weights
```

Alternating Training

```
For each training iteration:
  1. Sample real batch
  2. Sample noise batch
  3. Generate fake images
```

```
  4. Train D on real and fake (1 step or more)
  5. Generate new fake images
  6. Train G to fool D (1 step)
  7. Repeat
```

GAN Implementation

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Generator
def build_generator():
    model = keras.Sequential([
        layers.Dense(256, activation='relu', input_shape=(100,)),
        layers.Dense(512, activation='relu'),
        layers.Dense(1024, activation='relu'),
        layers.Dense(28*28*1, activation='sigmoid'),
        layers.Reshape((28, 28, 1))
    ])
    return model

# Discriminator
def build_discriminator():
    model = keras.Sequential([
        layers.Flatten(input_shape=(28, 28, 1)),
        layers.Dense(512, activation='relu'),
        layers.Dense(256, activation='relu'),
        layers.Dense(1, activation='sigmoid')
    ])
    return model

# Build GAN
generator = build_generator()
discriminator = build_discriminator()

# Compile discriminator
discriminator.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# GAN (generator + discriminator)
discriminator.trainable = False
gan_input = layers.Input(shape=(100,))
fake_images = generator(gan_input)
gan_output = discriminator(fake_images)

gan = keras.Model(gan_input, gan_output)
gan.compile(optimizer='adam', loss='binary_crossentropy')

# Training
epochs = 50
for epoch in range(epochs):
```

```
    # Train discriminator
    real_images = x_train[np.random.randint(0, x_train.shape[^0], 32)]
    noise = np.random.normal(0, 1, (32, 100))
    fake_images = generator.predict(noise)

    d_loss_real = discriminator.train_on_batch(real_images, np.ones((32, 1)))
    d_loss_fake = discriminator.train_on_batch(fake_images, np.zeros((32, 1)))

    # Train generator
    noise = np.random.normal(0, 1, (32, 100))
    g_loss = gan.train_on_batch(noise, np.ones((32, 1)))

    if epoch % 10 == 0:
        print(f'Epoch {epoch}: D_loss={d_loss_real+d_loss_fake:.4f}, G_loss={g_loss:.4f}'
```

GAN Challenges

1. Mode Collapse

Problem: Generator produces limited variety

```
Mode Collapse Example:
  Generator learns to generate single digit extremely well
  But ignores all other digits

  Discriminator can't tell it's the same digit
  Generator stops improving

  Result: Loss of diversity
```

Solutions:

- **Minibatch discrimination**: Discriminator considers whole batch
- **Feature matching**: Match feature statistics
- **Unrolled GANs**: Consider future discriminator steps
- **Ensemble**: Multiple generators

2. Vanishing Gradient

Problem: Early training, discriminator too good

```
Discriminator: Easily distinguishes real/fake
Gradient to generator: Near zero
Generator can't improve!

Symptom: G_loss plateaus, doesn't decrease
```

Solutions:

- **Wasserstein Loss**: Different loss function
- **Gradient penalty**: Penalize large gradients
- **Spectral normalization**: Stabilize discriminator
- **Better learning rates**: Careful tuning

3. Training Instability

Problem: Training oscillates, doesn't converge

```
Early epochs: Both networks learning rapidly
Mid epochs: Oscillations, unstable loss
Late epochs: Collapse or divergence
```

Solutions:

- **Label smoothing**: 0.9 instead of 1.0 for real
- **Noisy labels**: Add small noise
- **Batch normalization**: Stabilize layers
- **Lower learning rates**: More gradual updates

GAN Variants

DCGAN (Deep Convolutional GAN)

Innovation: Use convolutional layers for images

```
Generator:
  Transposed convolutions
  Better for upsampling

Discriminator:
  Regular convolutions
  Stable training

Result: High-quality image generation
```

Conditional GAN (cGAN)

Idea: Control generation with class label

```
Generator:
  Input: noise + class label
  Output: Generated image of that class

Discriminator:
  Input: image + class label
```

```
  Output: real/fake

Result: Can generate specific classes
```

CycleGAN

Idea: Unpaired image-to-image translation

```
Domain A (Horses) ←→ Domain B (Zebras)

No paired examples needed!

Process:
  1. Translate A → B
  2. Translate back B → A
  3. Should recover original

Cycle consistency loss ensures quality
```

StyleGAN

Idea: Control style and content separately

```
Architecture:
  Mapping Network: z → w (style vector)
  Synthesis Network: w → image (adaptive instance norm)

Result:
  - Control fine details independently
  - Photorealistic faces
  - Extreme quality
```

GAN Applications

1. Image Generation

```
Training: Real face images
Result: Generate new, realistic faces

Applications:
  - Character design
  - Avatar generation
  - Dataset augmentation
  - Synthetic data for training
```

2. Image-to-Image Translation

```
Input: Sketch
Output: Realistic image

Input: Day image
Output: Night image

Applications: Photo editing, style transfer
```

3. Super-Resolution

```
Input: Low-resolution image (32×32)
Output: High-resolution image (256×256)

Applications:
  - Medical imaging
  - Satellite imagery
  - Video enhancement
```

4. Data Augmentation

```
Problem: Limited training data
Solution: Generate synthetic training data

Example: 100 real medical images
         ↓ Train GAN
         ↓ Generate 10,000 synthetic
         ↓ Train diagnosis model on 10,100 images
         ↓ Much better performance!
```

Autoencoders vs VAE vs GAN Comparison

Aspect	Autoencoder	VAE	GAN
Architecture	Encoder-Decoder	Encoder-Decoder+Distribution	Generator-Discriminator
Training	Unsupervised	Unsupervised	Adversarial
Latent Space	Unstructured	Continuous, Gaussian	Unstructured
Generation	Limited	Good	Excellent
Training Stability	Stable	Stable	Unstable
Reconstruction	Sharp	Blurry	N/A
Best For	Compression, Denoising	Smooth generation	High-quality images
Computational Cost	Low	Medium	High

Data Compression with Autoencoders

Compression Pipeline

```
Original Image (28×28×1 = 784 bytes)
  ↓ Encode (Autoencoder)
  ↓ Latent Code (32 floats ≈ 128 bytes)
  ↓ Compression Ratio: 784/128 ≈ 6x

Decompression:
  Latent Code (128 bytes)
  ↓ Decode (Autoencoder)
  ↓ Reconstructed Image (784 bytes, ~95% quality)
```

Lossy vs Lossless

Lossless Compression:

- Can perfectly reconstruct original
- Limited compression ratio
- ZIP, PNG

Lossy Compression:

- Can't perfectly reconstruct
- Higher compression ratio
- JPEG, AutoEncoder compression
- Acceptable for images (human vision limitations)

Implementation

```
# Compress image
latent = encoder.predict(image)  # 32D vector

# Store/transmit
compressed = latent.tobytes()

# Decompress
latent_restored = np.frombuffer(compressed, dtype=np.float32)
reconstructed = decoder.predict(latent_restored)
```

Advanced Topics

Adversarial Training Techniques

Spectral Normalization:

```
Normalize discriminator weights
Constraint: ||W|| ≤ 1
Effect: Stabilize training
```

Gradient Penalty:

```
Add penalty for large gradients
L_penalty = λ * ||∇D||²
Effect: Smoother decision boundary
```

Progressive Growing:

```
Start with low resolution
Gradually increase resolution
Effect: Stable, high-quality generation
```

References

1. Kingma, D. P., & Welling, M. (2023). Auto-encoding variational Bayes. arXiv preprint arXiv:1312.6114.
2. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2023). Generative adversarial nets. In Advances in neural information processing systems (pp. 2672-2680).
3. Radford, A., Metz, L., & Chintala, S. (2023). Unsupervised representation learning with deep convolutional generative adversarial networks. arXiv preprint arXiv:1511.06434.
4. Zhu, J. Y., Park, T., Isola, P., & Efros, A. A. (2023). Unpaired image-to-image translation using cycle-consistent adversarial networks. In Proceedings of the IEEE International Conference on Computer Vision (pp. 2223-2232).
5. Karras, T., Aila, T., Laine, S., & Lehtinen, J. (2023). Progressive growing of GANs for improved quality, stability, and variation. In International Conference on Learning Representations.
6. Wasserstein GAN. (2023). arXiv preprint arXiv:1701.07957.
7. Spectral Normalization for GANs. (2023). arXiv preprint arXiv:1802.05957.
8. Conditional GAN. (2023). arXiv preprint arXiv:1411.1784.

PRACTICAL EXERCISE

Build Your Own Autoencoder

```
# 1. Prepare data
x_train, x_test = load_data()

# 2. Build autoencoder
autoencoder = build_autoencoder(latent_dim=32)

# 3. Train
autoencoder.fit(x_train, x_train, epochs=50)

# 4. Evaluate
reconstructed = autoencoder.predict(x_test)
mse = np.mean((x_test - reconstructed) ** 2)
print(f"Reconstruction MSE: {mse}")

# 5. Analyze latent space
encoder = autoencoder.layers[^0]  # Get encoder part
latent_features = encoder.predict(x_test)
visualize_latent_space(latent_features)
```

Experiment Ideas

1. **Compare Architectures**: Different encoder/decoder designs
2. **Compression Ratios**: Vary latent dimension size
3. **Anomaly Detection**: Use reconstruction error as anomaly score
4. **GAN Competition**: Generate images, compare with VAE

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19]

⁂

1. https://ai.google/principles/
2. https://arxiv.org/abs/2312.10997
3. https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api
4. https://www.promptingguide.ai/techniques
5. https://www.k2view.com/blog/prompt-engineering-techniques/
6. https://aws.amazon.com/blogs/machine-learning/prompt-engineering-techniques-and-best-practices-learn-by-doing-with-anthropics-claude-3-on-amazon-bedrock/
7. https://arxiv.org/html/2507.03405v1
8. https://ubiai.tools/the-complete-guide-to-generative-adversarial-networks-gans/
9. https://arxiv.org/html/2411.14511v1
10. https://dextralabs.com/blog/prompt-engineering-for-llm/
11. https://www.artificialintelligencepub.com/jairi/article/view/jairi-aid1004/5887
12. https://www.sciencedirect.com/science/article/pii/S2666827024000483

13. https://www.ibm.com/think/topics/generative-ai
14. https://aiverifyfoundation.sg/wp-content/uploads/2024/05/Model-AI-Governance-Framework-for-Generative-AI-May-2024-1-1.pdf
15. https://www.centraleyes.com/generative-ai-governance/
16. https://www.geeksforgeeks.org/nlp/transformer-attention-mechanism-in-nlp/
17. https://www.promptingguide.ai/research/rag
18. https://www.informationpolicycentre.com/uploads/5/7/1/0/57104281/cipl_applying_data_protection_principles_genai_dec24.pdf
19. https://arxiv.org/abs/1706.03762